À la fréquence de Fourier

1.1 La Transformée de Fourier

À quoi ça sert ?

Un monde parallèle ou les choses deviennent plus simples (ou plus compliquées)

- **1.2.1 Traitement du signal** Séparation des contributions de chaque fréquence
- **1.2.2 Avantage computationel** L'opération de convolution devient une multiplication classique

1.1.1 Illustration : séparation des fréquences

Audio

Error message

```
UndefVarError: CounterButton not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
@bind la_clicked CounterButton("play la")
```

👀 *Reading hidden code*

Error message

Another cell defining la_clicked contains errors.

```
let
    la_clicked

    échantillonnage = 2^13
    Δt = 1 / échantillonnage
    temps = range(Δt, stop=2, length=échantillonnage)
    la = cispi.(2*440*temps)
    play_sound(la, échantillonnage)
    nothing
end
```

👀 Reading hidden code

Error message

UndefVarError: `Button` not defined in this notebook.

Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
@bind ré_clicked Button("play ré")
```

👀 Reading hidden code

Error message

Another cell defining ré_clicked contains errors.

```
let
    ré_clicked

    échantillonnage = 2^13
    Δt = 1 / échantillonnage
    temps = range(Δt, stop=2, length=échantillonnage)
    ré = cispi.(2*293.7*temps)
    play_sound(ré, échantillonnage)
    nothing
end
```

👀 Reading hidden code

Error message

```
UndefVarError: Button not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
@bind la_ré_clicked Button("play la + ré")
```

👀 Reading hidden code

Error message

Another cell defining la_ré_clicked contains errors.

```
let
    la_ré_clicked

    échantillonnage = 2^13
    Δt = 1 / échantillonnage
    temps = range(Δt, stop=2, length=échantillonnage)
    la = cispi.(2*440*temps)
    ré = cispi.(2*293.7*temps)
    play_sound(la + ré, échantillonnage)
    nothing
end
```

👀 Reading hidden code

Signal temporel

Error message

Another cell defining échantillonnage contains errors.

```
md"Un son est un signal continu mais on va l'[échantillonner](https://fr.wikipedia.org/wiki/%C3%89chantillonnage_(signal)). C'est à dire qu'on va prendre un nombre fini de valeur par seconde à des distance égale dans le temps. Dans cet exemple, on prend une valeur toute les $(échantillonnage)ᵉ de seconde."
```

👀 Reading hidden code

Error message

```
UndefVarError: Slider not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"Fréquence d'échantillonnage = $(@bind échantillonnage Slider((2).^(4:13), default=1024, show_value = true))"
```

Reading hidden code

Ça correspond à une distance en seconde entre deux échantillons de:

Error message

```
Another cell defining échantillonnage contains errors.
```

```
Δt = 1 / échantillonnage
```

Error message

```
UndefVarError: Slider not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"nombre\_échantillons = $(@bind nombre_échantillons Slider((2).^(4:13), default=1024, show_value = true))"
```

Reading hidden code

Error message

> *Another cell defining nombre_échantillons or échantillonnage contains errors.*

```
temps_total = nombre_échantillons * Δt
```

👀 Reading hidden code

Error message

> *Another cell defining nombre_échantillons or échantillonnage contains errors.*

```
md"En prenant $(nombre_échantillons), le signal dure $(temps_total) secondes. Comme ces nombres sont équidistants (distance de `Δt` secondes entre eux), on peut représenter cette suite de façon compact comme suit:"
```

👀 Reading hidden code

Error message

> *Another cell defining nombre_échantillons or échantillonnage contains errors.*

```
temps = range(Δt, stop=temps_total, length=nombre_échantillons)
```

Le *la* est une note de musique de fréquence 440 Hz

Error message

> *Another cell defining nombre_échantillons or échantillonnage contains errors.*

```
la = cispi.(2*440*temps)
```

La note de musique *ré* a une fréquence de 293.7 Hz

Error message

> *Another cell defining nombre_échantillons or échantillonnage contains errors.*

```
ré = cispi.(2*293.7*temps)
```

Error message

> *Another cell defining échantillonnage contains errors.*

```
md"temps\_zoom = $(@bind temps_zoom Slider(range(Δt, stop=1000Δt, length=32),
default=10 / 440, show_value = true))"
```

👀 Reading hidden code

Error message

> *Another cell defining nombre_échantillons, échantillonnage, la, temps_zoom, or ré contains errors.*

```
begin
    index_max = something(findfirst(t -> t >= temps_zoom, temps), length(temps))
    plot(temps[1:index_max], real.(la[1:index_max]), label = "la")
    plot!(temps[1:index_max], real.(ré[1:index_max]), label = "ré")
    plot!(temps[1:index_max], real.(la[1:index_max] + ré[1:index_max]), label = "la
    + ré")
end
```

👀 Reading hidden code

Signal fréquentiel

Error message

> **UndefVarError: `Select` not defined in this notebook.**
> Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
@bind freq_sel Select(["abs", "real", "imag"])
```

👀 Reading hidden code

Error message

Another cell defining échantillonnage, nombre_échantillons, Δt, temps, ré, or freq_sel contains errors.

```
let
    f = range(0, stop = échantillonnage - 1/temps_total, length=length(la))
    sel = Dict(
        "abs" => abs,
        "real" => real,
        "imag" => imag,
    )[freq_sel]
    plot(f, sel.(fft(la)), label = "la", linewidth=5)
    plot!(f, sel.(fft(ré)), label = "ré", linewidth=5, color = :green)
    plot!(f, sel.(fft(la + ré)), label = "la + ré", linewidth=2, color = :red)
end
```

👀 Reading hidden code

1.1.2 Calcul rapide de convolution

La convolution continue:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(s)g(x-s)\mathrm{d}s$$

La convolution discrète:

$$(f * g)_n = \sum_{k=-\infty}^{\infty} f_k g_{n-k}\mathrm{d}s$$

Propriétés algébriques:

$$\begin{aligned}
\text{commutatif} \quad & f * g = g * f \\
\text{associatif} \quad & (f * g) * h = f * (g * h) \\
\text{distributif} \quad & f * (g + h) = f * g + f * h \\
\text{conjugué} \quad & \overline{f * g} = \overline{f} * \overline{g}
\end{aligned}$$

La cross-corrélation:

$$(f \star g)(x) = \int_{-\infty}^{\infty} f(s)g(x+s)\mathrm{d}s \qquad (f \star g)_n = \sum_{-\infty}^{\infty} f_k g_{n+k}\mathrm{d}s$$

Convolution theorem: Notons la tranformée de Fourier de f par $\mathcal{F}(f)$. Par la transformée de Fourier, la convolution devient un produit classique:

$$f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g))$$

▶ **Quel est la complexité de la convolution vs produit classique s'ils sont discrets de longueur n ?**

1.1.2.1 Illustration: convolution d'images

Error message

```
UndefVarError: testimage not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```python
mandrill = testimage("mandrill")
```

Gaussian Kernel

La fonction `gaussian(d)` donne une matrix de taille $(2d+1) \times (2d+1)$ contenant la valeur d'une Gaussienne.

Error message

```
UndefVarError: surface not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

```
surface(-2d:2d, -2d:2d, Kernel.gaussian(d))
```

Error message

```
UndefVarError: Slider not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"d = $(@bind d Slider(1:10, default=5, show_value = true))"
```

👀 *Reading hidden code*

La fonction `imfilter` calcule la cross-corrélation.

Error message

```
UndefVarError: imfilter not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

```
imfilter(mandrill, Kernel.gaussian(d))
```

La convolution est obtenue avec `reflect`.

Error message

UndefVarError: `imfilter` not defined in this notebook.

Suggestion: check for spelling errors or missing imports.

```
imfilter(mandrill, reflect(Kernel.gaussian(d)))
```

▶ **Pourquoi obtient-on le même résultat avec la convolution et la cross-corrélation ?**

Performance

```
imfilter_time (generic function with 1 method)
```

```
function imfilter_time(n, alg)
    A = rand(n, n)
    B = rand(n, n)
    return @elapsed imfilter(A, B, alg)
end
```

Error message

UndefVarError: `plot` not defined in this notebook.

Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
let
    n = 2 .^ (1:6)
    fir_time(n) = imfilter_time(n, Algorithm.FIR())
    fft_time(n) = imfilter_time(n, Algorithm.FFT())
    plot(n, fir_time ∘ Int, label = "Finite Impulse Response (FIR)")
    plot!(n, fft_time ∘ Int, label = "Fast Fourier Transform (FFT)")
end
```

Error message

```
UndefVarError: Kernel not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
convert(AbstractArray, Kernel.Laplacian())
```

Autre kernels

Error message

```
UndefVarError: imfilter not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

```
imfilter(mandrill, Kernel.Laplacian())
```

Différents kernels permettent d'analyser des aspects différents d'une image. Le kernel à utilisé peut aussi être **appris**, c'est la base des **Convolutional Neural Networks** (CNNs) !

1.1.2.2 Illustration : Le produit de polynômes

```
deg = 4
```

```
deg = 4
```

Error message

```
UndefVarError: Polynomial not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
p = Polynomial(rand(deg))
```

Error message

```
UndefVarError: Polynomial not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
q = Polynomial(rand(deg))
```

Error message

Another cell defining q or p contains errors.

```
p * q
```

Error message

Another cell defining Polynomials, q, or p contains errors.

```
Polynomials.poly_multiplication_fft(p, q)
```

```
poly_time (generic function with 1 method)
```

```julia
function poly_time(n, with_fft::Bool)
    p = Polynomial(rand(n))
    q = Polynomial(rand(n))
    @elapsed if with_fft
        Polynomials.poly_multiplication_fft(p, q)
    else
        p * q
    end
end
```

Error message

```
UndefVarError: plot not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
let
    n = 2 .^ (1:13)
    std_time(n) = poly_time(n, false)
    fft_time(n) = poly_time(n, true)
    plot(n, std_time ∘ Int, label = "Standard multiplication")
    plot!(n, fft_time ∘ Int, label = "With FFT")
end
```

▶ **Comment utiliser cela pour calculer le produit de grand nombres ?**

1.1.2 Définition de la transformée de Fourier

La transformée de Fourier continue et son inverse:

$$F(\xi) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi\xi t} \mathrm{d}t$$
$$f(t) = \int_{-\infty}^{\infty} F(\xi) e^{i2\pi\xi t} \mathrm{d}\xi$$

La transformée de Fourier discrète et son inverse:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi \frac{k}{N} n}$$

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi \frac{k}{N} n}$$

Rajouter linearité : TODO

▶ **Que vaut la distance entre deux valeurs successive en temporel et en fréquentiel pour les signaux discrets ?**

TODO : même dimension en temporel et fréquentiel

▶ **Que vaut le signal en dehors de ses N points ?**

1.1.3 Le replis spectral

On vient de voir que le signal est extrapolé périodiquement par la transformée de fourier discrète. Si on prend un signal de `0` à `t_max` et qu'on divise la fréquence d'échantillonnage de sa transformée de fourier par 2. Ça signifie que le signal sera 2 fois moins long.

▶ **Que va-t-il advenir de la deuxième moitié du signal ?**

Illustrons cela avec la fonction `exp(-(t-centre)^2)` qui est initiallement échantillonnée avec `512` échantillons de `0` à `t_max` et qu'on ré-échantillonne ensuite à `512 / ratio` en fréquentiel, ce qui divise `t_max` par `ratio` également.

Error message

```
UndefVarError: Slider not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
1 md"t\_max = $(@bind t_max Slider(range(1, stop=10, length=10), default=5,
  show_value = true))"
```

Reading hidden code

Error message

Another cell defining t_max contains errors.

```
md"centre = $(@bind centre Slider(range(0.1, stop=t_max, length=32), default=t_max / 2, show_value = true))"
```

👀 Reading hidden code

Error message

UndefVarError: `Slider` not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
md"ratio = $(@bind ratio Slider(2 .^ (0:4), default=2, show_value = true))"
```

👀 Reading hidden code

Error message

Another cell defining ratio, t_max, or centre contains errors.

```
let
    N = 512
    d = ratio
    x = range(0, stop = t_max - t_max / N, length = N)
    gauss(x) = exp(-16(x-centre)^2)/4 + exp(-16(x-centre/2)^2)
    plot(x, gauss, label = "signal original")
    X = fft(gauss.(x))
    Xs = X[1:d:N]
    half = div(N, 2d)
    xs = real.(ifft(Xs))
    plot!(x[1:length(xs)], xs, label = "rééchantilloné")
end
```

👀 Reading hidden code

1.2 Fast Fourier Transform

Attardons-nous à présent sur l'algorithme utilisé pour calculé la DFT (transformée de fourier discrète). On a vu précédemment que cet algorithme a une complexité de $\Omega(n\log(n))$. C'est parmis le top 10 des meilleurs algorithmes du 20e siècle ! Son Implémentation la plus rapide est dans la libraire FFTW : the Fastest Fourier Transform in the West ! Comment est-elle la plus rapide ? Voir ici. L'algorithme est initialement découvert part Gauss en 1805 puis redécouvert par Cooley et Tukey en 1965.

La DFT est en fait une évaluation d'un polynômes aux différentes racines `N` ièmes de l'unité:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi\frac{k}{N}n}$$
$$X_k = \sum_{n=0}^{N-1} x_n z_{k,N}^n \qquad z_{k,N} = e^{-i2\pi\frac{k}{N}}$$

La DFT peut aussi être vue comme un produit matriciel:

$$X = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 \\ 1 & z_N & \cdots & z_N^{N-2} & z_N^{N-1} \\ 1 & z_N^2 & \cdots & z_N^{2(N-2)} & z_N^{2(N-1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & z_N^{(N-1)2} & \cdots & z_N^{(N-1)(N-2)} & z_N^{(N-1)(N-1)} \end{bmatrix} x$$

Un produit matriciel a complexité $\Omega(n^2)$. Ça ne nous donne pas une complexité de $\Omega(n\log n)$, il va falloir utiliser la structure assez spéciale de cette matrice...

On a acquis une intuition géométrique sur les racines z_N. On va s'en servir pour visualiser cette matrice.

```
8×8 Matrix{ComplexF64}:
 1.0+0.0im        1.0+0.0im        1.0+0.0im  …   1.0+0.0im        1.0+0.0im
 1.0+0.0im   0.707107-0.707107im   0.0-1.0im      0.0+1.0im   0.707107+0.707107im
 1.0+0.0im        0.0-1.0im       -1.0-0.0im     -1.0-0.0im        0.0+1.0im
 1.0+0.0im  -0.707107-0.707107im   0.0+1.0im      0.0-1.0im  -0.707107+0.707107im
 1.0+0.0im       -1.0-0.0im        1.0-0.0im      1.0-0.0im       -1.0-0.0im
 1.0+0.0im  -0.707107+0.707107im   0.0-1.0im  …   0.0+1.0im  -0.707107-0.707107im
 1.0+0.0im        0.0+1.0im       -1.0-0.0im     -1.0-0.0im        0.0-1.0im
 1.0+0.0im   0.707107+0.707107im   0.0+1.0im      0.0-1.0im   0.707107-0.707107im
```

```
F(8)
```

▶ **Qu'observe-t-on dans la matrice ci-dessous ?**

Error message

> `UndefVarError: CheckBox not defined in this notebook.`
> `Suggestion: check for spelling errors or missing imports.`

Show stack trace...

```
md"Besoin d'un indice ? $(@bind colored_arrows CheckBox())"
```

Reading hidden code

Error message

> *Another cell defining colored_arrows contains errors.*

```
arrows(8, colored_arrows)
```

On peut le vérifier numériquement également:

```
partie_mauve = 4×4 Matrix{ComplexF64}:
                1.0+0.0im   1.0+0.0im   1.0+0.0im   1.0+0.0im
                1.0+0.0im   0.0-1.0im  -1.0-0.0im   0.0+1.0im
                1.0+0.0im  -1.0-0.0im   1.0-0.0im  -1.0-0.0im
                1.0+0.0im   0.0+1.0im  -1.0-0.0im   0.0-1.0im
```

```
partie_mauve = F(8)[1:4, 1:2:8]
```

```
partie_bleue = 4×4 Matrix{ComplexF64}:
                1.0+0.0im   1.0-0.0im   1.0-0.0im   1.0-0.0im
                1.0+0.0im   0.0-1.0im  -1.0-0.0im   0.0+1.0im
                1.0+0.0im  -1.0-0.0im   1.0-0.0im  -1.0-0.0im
                1.0+0.0im   0.0+1.0im  -1.0-0.0im   0.0-1.0im
```

```
partie_bleue = F(8)[5:8, 1:2:8]
```

```
true
```

```
partie_mauve == partie_bleue
```

```
partie_verte = 4×4 Matrix{ComplexF64}:
                      1.0+0.0im             1.0+0.0im       …        1.0+0.0im
                 0.707107-0.707107im  -0.707107-0.707107im      0.707107+0.707107im
                      0.0-1.0im             0.0+1.0im               0.0+1.0im
                -0.707107-0.707107im   0.707107-0.707107im     -0.707107+0.707107im
```

```
partie_verte = F(8)[1:4, 2:2:8]
```

```
partie_rouge = 4×4 Matrix{ComplexF64}:
                     -1.0-0.0im            -1.0-0.0im       …       -1.0-0.0im
                -0.707107+0.707107im   0.707107+0.707107im     -0.707107-0.707107im
                      0.0+1.0im             0.0-1.0im               0.0-1.0im
                 0.707107+0.707107im  -0.707107+0.707107im      0.707107-0.707107im
```

```
partie_rouge = F(8)[5:8, 2:2:8]
```

```
true
```

```
partie_verte == -partie_rouge
```

```
true
```

```
isapprox(0.0, 1e-300, atol = 1e-8)
```

▶ **Qu'observe-t-on en comparant cela à la matrice 2 fois plus petite ?**

Error message

UndefVarError: plot not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. **arrows**(n::Int64, use_color::Bool)
   from Other cell: *line 2*

```
function arrows(n, use_color = false)
 plot(showaxis = false)
 for i in 0:(n-1)
     for j in 0:(n-1)
```

cell preview

2. Show more...

```
arrows(4, false)
```


Error message

UndefVarError: plot not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. **arrows**(n::Int64, use_color::Bool)
   from Other cell: *line 2*

```
function arrows(n, use_color = false)
 plot(showaxis = false)
 for i in 0:(n-1)
     for j in 0:(n-1)
```

cell preview

2. Show more...

```
arrows(8, true)
```

👀 Reading hidden code

Vérifions le à nouveau numériquement:

```
4×4 Matrix{ComplexF64}:
 1.0+0.0im   1.0-0.0im   1.0-0.0im   1.0-0.0im
 1.0+0.0im   0.0-1.0im  -1.0-0.0im   0.0+1.0im
 1.0+0.0im  -1.0-0.0im   1.0-0.0im  -1.0-0.0im
 1.0+0.0im   0.0+1.0im  -1.0-0.0im   0.0-1.0im
```

```
partie_bleue
```

```
4×4 Matrix{ComplexF64}:
 1.0+0.0im   1.0+0.0im   1.0+0.0im   1.0+0.0im
 1.0+0.0im   0.0-1.0im  -1.0-0.0im   0.0+1.0im
 1.0+0.0im  -1.0-0.0im   1.0-0.0im  -1.0-0.0im
 1.0+0.0im   0.0+1.0im  -1.0-0.0im   0.0-1.0im
```

```
F(4)
```

```
true
```

```
partie_bleue == F(4)
```

```
4×4 Matrix{ComplexF64}:
       1.0+0.0im             1.0+0.0im       …        1.0+0.0im
  0.707107-0.707107im  -0.707107-0.707107im      0.707107+0.707107im
       0.0-1.0im             0.0+1.0im               0.0+1.0im
 -0.707107-0.707107im   0.707107-0.707107im     -0.707107+0.707107im
```

```
partie_verte
```

```
0.7071067811865476 - 0.7071067811865475im
```

```
cispi(-2/8)
```

```
▸[1.0+0.0im, 0.707107-0.707107im, 2.22045e-16-1.0im, -0.707107-0.707107im]
```

```
cispi(-2/8) .^ collect(0:3)
```

```
4×4 Matrix{ComplexF64}:
         1.0+0.0im                1.0+0.0im       …           1.0+0.0im
    0.707107-0.707107im     -0.707107-0.707107im         0.707107+0.707107im
 2.22045e-16-1.0im       -2.22045e-16+1.0im          -2.22045e-16+1.0im
   -0.707107-0.707107im      0.707107-0.707107im        -0.707107+0.707107im
```

```
cispi(-2/8) .^ (0:3) .* F(4)
```

```
4×4 BitMatrix:
 1  1  1  1
 1  1  1  1
 1  1  1  1
 1  1  1  1
```

```julia
isapprox.(real.(partie_verte), real.(cispi(-2/8) .^ (0:3) .* F(4)), atol=1e-8)
```

En se rappelant que l'intuition géométrique des `N` racines de l'unité, on comprend pourquoi les `N/2` racines sont un sous-ensembles des `N` racines.

Error message

UndefVarError: `Slider` not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Show stack trace...

```julia
md"`max_N` = $(@bind max_N Slider(2 .^ (2:5), default=8, show_value = true))"
```

Reading hidden code

Error message

UndefVarError: `plot` not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

```julia
let
    p = plot(ratio = :equal, ticks = nothing, showaxis = false)
    max_pow = round(Int, log2(max_N))
    colors = []
    for pow in max_pow:-1:2
        draw_roots!(2^pow, linewidth = 2 * (pow - 1), color =
        Colors.JULIA_LOGO_COLORS[pow - 1])
    end
    p
end
```

Reading hidden code

Utilitaires

Error message from Main

The package Plots.jl could not load because it failed to initialize.

That's not nice! Things you could try:

- Restart the notebook.
- Try a different Julia version.
- Contact the developers of Plots.jl about this error.

You might find useful information in the package installation log:

```
      @ stdin:5
   [20] eval(m::Module, e::Any)
      @ Core ./boot.jl:489
   [21] include_string(mapexpr::typeof(identity), mod::Module, code::String, filename::String)
      @ Base ./loading.jl:2842
   [22] include_string
      @ ./loading.jl:2852 [inlined]
   [23] exec_options(opts::Base.JLOptions)
      @ Base ./client.jl:315
   [24] _start()
      @ Base ./client.jl:550
  in expression starting at /home/runner/.julia/packages/Images/Kttvj/src/Images.jl:1
  in expression starting at stdin:
```

```julia
using Plots
```

Error message from Main

The package Plots.jl could not load because it failed to initialize.

That's not nice! Things you could try:

- Restart the notebook.
- Try a different Julia version.
- Contact the developers of Plots.jl about this error.

You might find useful information in the package installation log:

```
      @ stdin:5
   [20] eval(m::Module, e::Any)
      @ Core ./boot.jl:489
   [21] include_string(mapexpr::typeof(identity), mod::Module, code::String, filename::String)
      @ Base ./loading.jl:2842
   [22] include_string
      @ ./loading.jl:2852 [inlined]
   [23] exec_options(opts::Base.JLOptions)
      @ Base ./client.jl:315
   [24] _start()
      @ Base ./client.jl:550
  in expression starting at /home/runner/.julia/packages/Images/Kttvj/src/Images.jl:1
  in expression starting at stdin:
```

```julia
using PlutoUI, FFTW, PortAudio, Images, TestImages, ImageFiltering
```

Error message from Main

The package Plots.jl could not load because it failed to initialize.

That's not nice! Things you could try:

- Restart the notebook.
- Try a different Julia version.
- Contact the developers of Plots.jl about this error.

You might find useful information in the package installation log:

```
      @ stdin:5
   [20] eval(m::Module, e::Any)
      @ Core ./boot.jl:489
   [21] include_string(mapexpr::typeof(identity), mod::Module, code::String, filename::String)
      @ Base ./loading.jl:2842
   [22] include_string
      @ ./loading.jl:2852 [inlined]
   [23] exec_options(opts::Base.JLOptions)
      @ Base ./client.jl:315
   [24] _start()
      @ Base ./client.jl:550
  in expression starting at /home/runner/.julia/packages/Images/Kttvj/src/Images.jl:1
  in expression starting at stdin:
```

```julia
using Polynomials, LinearAlgebra
```

play_sound (generic function with 1 method)

```julia
function play_sound(sound, samplerate)
    # For the static version, there is no sound so we just make the buttons
    # do nothing
    if PortAudio.Pa_GetDefaultOutputDevice() != PortAudio.LibPortAudio.paNoDevice
        PortAudioStream(0, 2; samplerate) do stream
            write(stream, real.(sound))
        end
    end
end
```

```julia
begin
    struct Join
        list
        Join(a) = new(a)
        Join(a, b, args...) = Join(tuple(a, b, args...))
    end
    function Base.show(io::IO, mime::MIME"text/html", d::Join)
        for el in d.list
            show(io, mime, el)
        end
    end
end
```

```julia
begin
    struct HTMLTag
        tag::String
        parent
    end
    function Base.show(io::IO, mime::MIME"text/html", d::HTMLTag)
        write(io, "<", d.tag, ">")
        show(io, mime, d.parent)
        write(io, "</", d.tag, ">")
    end
end
```

```
f (generic function with 1 method)
```

```julia
f(i, j, n) = cispi(-2 * i * j / n)
```

```
F (generic function with 1 method)
```

```julia
function F(n)
    A = [
        f(k, m, n)
        for k in 0:(n-1), m in 0:(n-1)
    ]
end
```

```
8×8 Matrix{ComplexF64}:
 1.0+0.0im        1.0+0.0im        1.0+0.0im  …   1.0+0.0im        1.0+0.0im
 1.0+0.0im   0.707107-0.707107im   0.0-1.0im      0.0+1.0im   0.707107+0.707107im
 1.0+0.0im        0.0-1.0im       -1.0-0.0im     -1.0-0.0im        0.0+1.0im
 1.0+0.0im  -0.707107-0.707107im   0.0+1.0im      0.0-1.0im  -0.707107+0.707107im
 1.0+0.0im       -1.0-0.0im        1.0-0.0im      1.0-0.0im       -1.0-0.0im
 1.0+0.0im  -0.707107+0.707107im   0.0-1.0im  …   0.0+1.0im  -0.707107-0.707107im
 1.0+0.0im        0.0+1.0im       -1.0-0.0im     -1.0-0.0im        0.0-1.0im
 1.0+0.0im   0.707107+0.707107im   0.0+1.0im      0.0-1.0im   0.707107-0.707107im
```

```julia
F(8)
```

```
arrows (generic function with 2 methods)
```

```julia
function arrows(n, use_color = false)
    plot(showaxis = false)
    for i in 0:(n-1)
        for j in 0:(n-1)
            if use_color
                color = Colors.JULIA_LOGO_COLORS[1 + iseven(j) * 2 + (2i < n)]
            else
                color = :black
            end
            scatter!([j], [-i], markersize = 8 / sqrt(n), markerstrokewidth = 0, legend = nothing; color)
            c = f(i, j, n) / 3 # Divide by 3 so that arrow is not too long in plot
            plot!([j, j + real(c)], [-i, -i + imag(c)], arrow = true, legend = nothing; color)
        end
    end
    plot!()
end
```

```
draw_roots! (generic function with 1 method)
```

```julia
function draw_roots!(N; kws...)
    for i in 1:N
        b, a = sincospi(2 * i / N)
        plot!([0, a], [0, b]; arrow = true, label = nothing, kws...)
    end
end
```

```
qa (generic function with 1 method)
```

```julia
function qa(question, answer)
    return HTMLTag("details", Join(HTMLTag("summary", question), answer))
end
```